VANDERBILT UNIVERSITY

AI Days
3/5/2026

Specification-Driven Development
Building Research Tools Without Coding

Myranda Uselton Shirk *(they/them)*

Senior Data Scientist and Lecturer

Session Roadmap

1. Why Specs Matter — 15 min
2. The Workflow — 10 min
3. Example Cases — 30–45 min
4. Testing & Evaluation — 15 min

Specifications and Software Engineering

- High-level:
  - Who is the user?
  - What problem does the software solve?
  - What does the software need to do?
  - Quality testing
- Low-level:
  - Programming environment?
  - Algorithms?
  - Error handling

What We Assumed vs. What's True Now

The Old Assumption

- You need **to code** to build research tools
- **Programming** is the bottleneck
- **AI** does all the work
- **Model** quality → Output quality

The New Reality

- You need to **specify** research needs
- **Communication** is the bottleneck
- **You** direct agents with precision
- **Specification** quality → Output

What We Assumed vs. What's True Now

The Old Assumption

- You need **to code** to build research tools
- **Programming** is the bottleneck
- **AI** does all the work
- **Model** quality → Output quality

The New Reality

- You need to **specify** research needs
- **Communication** is the bottleneck
- **You** direct agents with precision
- **Specification** quality → Output

Domain **expertise** + **specs**
= powerful **tools**

Term Definitions

- **Model** – a trained neural network/large language model (Claude 4.0)

  text in → text out
- **Agent** – framework for taking actions (Claude Code)

  model + control loop + memory + tools
- **Skills** – capabilities/tools the agent can invoke (read file, search web, etc)

Term Definitions

- **Model** – a trained neural network/large language model (Claude 4.0)

  text in → text out
- **Agent** – framework for taking actions (Claude Code)

  model + control loop + memory + tools
- **Skills** – capabilities/tools the agent can invoke (read file, search web, etc)

You write a **spec** → hand it to an **agent** → the agent uses its **model** + **skills** to build your tool

What Is a Specification?

- **Specification**: precise, testable description of what a tool should do — not how it should do it.

- 1. What **problem** does this tool solve?
- 2. What should be **input**? What should be **output**?
- 3. What **rules** must always hold?
- 4. How will I know if it’s **working**?

Anatomy of a Good Specification

1. Goal

One sentence: what problem does this solve?

Extract key themes from interview transcripts without me having to read manually.

2. Inputs/Outputs

What data, files, or parameters does it accept?

series of transcript files (mostly .docx)

What does it produce? In what format?

.csv of themes from each transcript

Anatomy of a Good Specification

4. Constraints

What rules must always be true?

Every transcript should be processed and included in the csv

The csv must include the columns: filename, date, theme, subtheme

5. Acceptance Criteria

How will you know it works under all conditions? How do you test it?

Write test specifications BEFORE building.

Given 5 examples I've already coded by hand, it matches mine 90% of the time

If given a .pdf file, it converts to .docx without losing information

Vague Request vs. Strong Specification

Vague Request

“Make a chart of my survey data”

“Process my interview transcripts”

Strong Specification

Extract key themes from interview transcripts according to the given rubric. Take in a file and output a CSV file with rows for each interview, with columns filename, date, theme, subtheme. The tool should be able to handle docx and pdf files, and should match the given examples

Vague Request vs. Strong Specification

Vague Request

“Make a chart of my survey data”

“Process my interview transcripts”

Domain **expertise** + **specs**
= powerful **tools**

Strong Specification

Extract key themes from interview transcripts according to the given rubric. Take in a file and output a CSV file with rows for each interview, with columns filename, date, theme, subtheme. The tool should be able to handle docx and pdf files, and should match the given examples

The Spec-Driven Workflow

1. **Specify — write a complete, precise specification**
2. **Direct — hand the spec to your coding agent**
3. **Test — run the tool against your sample data**
4. **Evaluate — check output against acceptance criteria**
5. **Refine — update the spec based on what you learned**

↺ Repeat until all acceptance criteria pass

What to Include

Your full specification in a document

The language/environment you're using

A sample of your input data

What "done" looks like (acceptance criteria)

Any constraints (libraries, output format)

What NOT to do

Don't describe the implementation ("use a for loop…")

Don't ask for multiple tools at once

Don't skip the acceptance criteria

Don't accept output without testing

Don't re-prompt without updating the spec

What Agents Need From You

- **1. Clarity of Intent**
- What is this tool for? Who uses it? What decision does it enable?

- **2. Concrete Examples**
- Show a sample input and the expected output.

- **3. Testable Criteria**
- How will we both know when it’s right?
- Write the tests first, not the tool.

Example Cases

Case Study: Medical Scheduling Agent

- **Client:** SOS Care Now — Chicago home healthcare
- **Developer:** DSI Team led by Umang Chaudry
- **Problem**
- Coordinators matched patients to providers and appointment times manually — slow and error-prone

Case Study: Medical Scheduling Agent

- **The Tool**
- LangGraph agent + Streamlit: matches patients to providers by availability
- Integrations: Google Calendar · Google Maps · Gmail · HuggingFace Spaces
- Not area-specific (can be used as a template for any provider)

Case Study: The Specification

- **Goal**: create a **template repo** for this project with everything needed to build the tool — correct structure, signatures, docstrings, TODOs
- Inputs → Outputs: business rules + provider data → 20+ files, 3 API integrations

- **The key constraint:**
- "Avoid hardcoding Chicago-specific values outside config.py"
- One sentence — shaped the entire architecture

- **Acceptance Criteria: 10-item checklist — binary, testable, runnable**

Overview

Create a complete project scaffold for a Medical Scheduling Agent. This is a **template repo** — every file should exist with the correct structure, imports, class/function signatures, docstrings, and TODO comments, but **not full implementations**. The student will complete the code.

The app is a Streamlit-based dashboard for SOS Care Now, a Chicago-area home healthcare practice, that matches patients to providers based on availability and location. It uses a LangGraph agent orchestrated with GPT-5.2 (OpenAI) and integrates with Google Calendar, Google Maps, and Gmail.

Deployment target: HuggingFace Spaces (Streamlit).

Directory Structure

Create exactly this file tree:

```
medical-scheduling-agent/
├── .gitignore
├── README.md
├── requirements.txt
├── app.py                          # Streamlit entry point (multipage)
├── config.py                       # Constants, visit levels, defaults
```

Notes for the Student

Add a `STUDENT_NOTES.md` file with the following content:

```
# Student Implementation Notes

## Project Context

This scheduling agent is being built for **SOS Care Now**, a home healthcare practice operating in the Chicago metro area. It i

Once the Chicago instance is complete, a second instance will be forked for SOS Care Now's Florida operations. To make that eas

## Getting Started

1. Read through the full codebase — every file has docstrings and TODO comments explaining what to build.
2. Start with **Section 1** and work sequentially. Each section builds on the previous.
3. The 5 providers in `data/providers.json` are real SOS Care Now staff. Their `lat`/`lng` fields are `null` and need to be geo

## Implementation Order

### Section 1: Data Layer
- `config.py` (already complete)
- `core/data_manager.py` — implement all CRUD functions
```

Final Checklist

Before marking the scaffold complete, verify:

- [ ] All files in the directory tree exist
- [ ] All JSON seed files have valid JSON
- [ ] All Python files parse without syntax errors (`python -c "import ast; ast.parse(open('file.py').read())"`)
- [ ] All TODO comments are in place
- [ ] All docstrings accurately describe what needs to be built
- [ ] `config.py` has actual values, not placeholders
- [ ] `agent/graph.py` has the actual graph structure, not just a TODO
- [ ] `agent/state.py` has the actual TypedDict, not just a TODO
- [ ] `requirements.txt` lists all needed packages
- [ ] README has complete setup instructions
- [ ] STUDENT_NOTES.md has clear implementation order

Using the build specs

1. **Save specs to build instruction (BUILD_INSTRUCTIONS.md)**
   1,000+ words — all 5 spec components explicit
2. **Provide to Claude Code**
   Agent scaffolds 20+ files/folders in one session
3. **Run the Final Checklist**
   Every Python file parses; imports resolve; no placeholders remain
4. **Show the output**
   agent/graph.py and data_manager.py — correct signatures, TODOs in place

Building the tool

- Clearly-marked areas for programmer to fill in as needed
- Ability to clone project and make specific to another location
- Build instructions provides documentation on the project
- Want to expand the tool? Write a spec for it!

Lessons Learned

- **One constraint, one architecture**
- “Avoid hardcoding” shaped the whole design — easy for someone to create a custom tool for another location

- **The spec served three audiences**
- Directed the agent + onboarded the student + documented for the client

Case Study: Digital Nashville

- **Developer:** DS team led by Myranda Shirk + Vanderbilt Law Lab

- **Problem**
- Nashville needs to prepare its infrastructure and policies for extreme heat events

Case Study: Digital Nashville

- **The Tool**
- A simulation of Nashville given different heat scenarios
- Models the effect on electricity, water, and transportation sectors
- Could serve as a template for other cities

Case Study: The Specification

- **Goal**: create a **build file** to simulate Nashville with the data available
- Inputs → Outputs: Nashville specific data + domain-informed modeling → web app simulating effects of heat on Nashville sectors (electricity, water, transportation)

- **Acceptance Criteria: checklist for files, libraries, JSON, data**

Testing & Evaluation

Test-Driven Development with Agents

- **Write your tests before asking the agent to build the tool.**

- Clarifies your own thinking before you start
- Gives the agent clear, measurable success criteria
- Makes evaluation mechanical, not subjective
- **You write test descriptions — the agent writes both tool AND tests**

Three Levels of Tests

- **Level 1: Smoke Test**
- Does it run? Example: "Run on sample data. Should complete without errors."

- **Level 2: Correctness Test**
- Right output? Example: "Given 10 rows, output should have exactly 10 bars."

- **Level 3: Edge Case Test**
- Bad inputs? Example: "If keywords file is empty, exit with a clear error."

Evaluating Agent Output

Four Questions to Ask

Does the output match the specification?

Does it pass all acceptance criteria?

Is it readable and maintainable?

Does it handle edge cases?

Red Flags

Output that “looks right” but was **never tested**

Agent added **features you didn’t ask for**

Acceptance criteria were **never checked**

Works on sample data but **fails on real data**

You can’t explain what the output does

When the Agent Gets It Wrong: Update the Spec

- Don’t just re-prompt — ask: what was missing in my spec?

- 1. **Identify the gap** — was the spec unclear or missing a constraint?
- 2. **Add a concrete example** — examples fix most spec failures
- 3. **Strengthen acceptance criteria** — make the test more specific
- 4. **Re-direct with the updated spec**

- Every failed output is feedback on your specification.

Common Pitfalls

- Writing specs that describe implementation, not behavior
- Skipping acceptance criteria entirely
- Asking for too much in one specification
- Accepting output without testing it
- Not updating the spec when requirements change
- Assuming the agent knows your domain — be explicit
- Asking the agent to do something you know how to do better

Key Takeaways

1. **Specification writing** is the core skill — **not coding**
2. **The loop never ends:** Specify → Direct → Test → Refine
3. **Tests before tools** — write acceptance criteria first
4. **Quality α specification** — be precise about what you want
5. **You are the expert** — agents execute your vision

Resources & Next Steps

Practice: Write a spec for a tool you actually need

- Pick something small — a 30-minute task, not a 3-month project
- Test it before polishing it
- When you get stuck: the spec is usually missing a constraint or example

Q&A

- What’s the hardest part of writing a clear specification?
- Where in your research workflow would this approach help most?
- What tool would you build if you could build anything?

VANDERBILT UNIVERSITY

Contact

myranda.shirk@vanderbilt.edu

Slides created with help of Claude Code, refined for content accuracy by Myranda Shirk